so time and space complexity could be considered for an algorithmic efficiency.

4.3.1 Method for determining Efficiency

The efficiency of an algorithm depends on how efficiently it uses time and memory space.

The time efficiency of an algorithm is measured by different factors. For example, write a program for a defined algorithm, execute it by using any programming language, and measure the total time it takes to run. The execution time that you measure in this case would depend on a number of factors such as:

- Speed of the machine
- Compiler and other system Software tools
- Operating System
- Programming language used
- Volume of data required

However, to determine how efficiently an algorithm solves a given problem, you would like to determine how the execution time is affected by the nature of the algorithm. Therefore, we need to develop fundamental laws that determine the efficiency of a program in terms of the nature of the underlying algorithm.

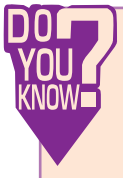
A way of designing algorithm is called algorithmic strategy

4.3.2 Space-Time tradeoff

A space-time or time-memory tradeoff is a way of solving in less time by using more storage space or by solving a given algorithm in very little space by spending more time.

To solve a given programming problem, many different algorithms may be used. Some of these algorithms may be extremely time-efficient and others extremely space-efficient.

Time/space trade off refers to a situation where you can reduce the use of memory at the cost of slower program execution, or reduce the running time at the cost of increased memory usage.



The best algorithm to solve a given problem is one that requires less space in memory and takes less time to execute its instructions to generate output.

4.3.3 Asymptotic Notations

Asymptotic Notations are languages that uses meaningful statements about time and space complexity. The following three asymptotic notations are mostly used to represent time complexity of algorithms:

(i) Big O

Big O is often used to describe the worst-case of an algorithm.

(ii) Big Ω

Big Omega is the reverse Big O, if Bi O is used to describe the upper bound (worst - case) of a asymptotic function, Big Omega is used to describe the lower bound (best-case).